Thermal-Aware Scheduling for Edge Inference

Authors: Ines Duarte and Kofi Asante (fictional authors; synthetic evaluation paper).

Identifier: doi:10.5555/ras-eval.p07 (fictional, not registered).

Abstract

We schedule neural network inference on edge devices using on-chip temperature forecasts. The scheduler cuts p95 latency by 23% and thermal throttling events from 41 to 9 per hour.

Introduction

Edge devices lower their clock speed when the chip gets too hot, so latency becomes erratic. Reactive governors respond only after throttling has started and cannot plan ahead. Inference workloads are often bursty, which makes short-term temperature spikes common. A scheduler that predicts temperature can delay low-priority jobs before throttling begins. Such planning must stay cheap, because the forecaster runs on the same constrained device. We therefore restrict the forecaster to a linear model that needs no accelerator support. The goal is predictable latency for interactive requests rather than maximum throughput. We compare against first-come-first-served scheduling with the default thermal governor. Our contribution is a scheduler that uses forecasts, together with measurements on devices.

Methods

The forecaster is a linear model over the last 20 temperature readings sampled at 10 Hz.
Jobs are deferred when the forecast exceeds 78 degrees Celsius within the next two seconds.
Deferred jobs wait in a priority queue and are resumed once the forecast falls again.
Interactive requests are never deferred for longer than 300 milliseconds in this design.
We evaluated three device types: a phone SoC, a single-board computer and a drone controller.
Each device ran the same image classification model with a batch size of one request.
Every experiment lasted two hours at an ambient temperature of 24 degrees Celsius.
Latency was logged per request, and throttling events were read from the kernel log.

Results

Across the three devices, p95 latency fell by 23% relative to first-come-first-served.

Throttling events dropped from 41 to 9 per hour on average across the three devices.

Energy per inference rose by 4% because deferred jobs later ran at higher clock speeds.

The phone SoC showed the largest latency gain, a 31% reduction of p95 latency.

The single-board computer improved by 27%, with most gains during long bursts.

Median latency changed by less than 2% on every device, so the gain lies in the tail.

Forecast error stayed below 1.5 degrees Celsius for 95% of the prediction windows.

The scheduler itself used less than 1% of processor time on each of the devices.

Discussion

The drone controller benefited least, with a 9% latency reduction.

Its cooling was already adequate, so the chip rarely approached the threshold.

Longer forecast horizons made the scheduler defer too many jobs in our trials.

A threshold below 78 degrees Celsius reduced throttling further but hurt throughput.

Results may differ for workloads with large batches or for devices with active fans.

We did not study multi-tenant settings where several models share one device.

Integrating the forecaster into the kernel governor is left for future work.

The measurements cover a single ambient temperature, which limits generality.

Conclusion

Forecast-driven deferral keeps tail latency predictable on thermally limited devices. The approach needs only a small linear model and a priority queue per device.

References

[1] P. Laine. Dynamic Thermal Management for Mobile Processors. 2023.

[2] G. Ortiz. Tail Latency on Embedded Inference Hardware. 2024.